

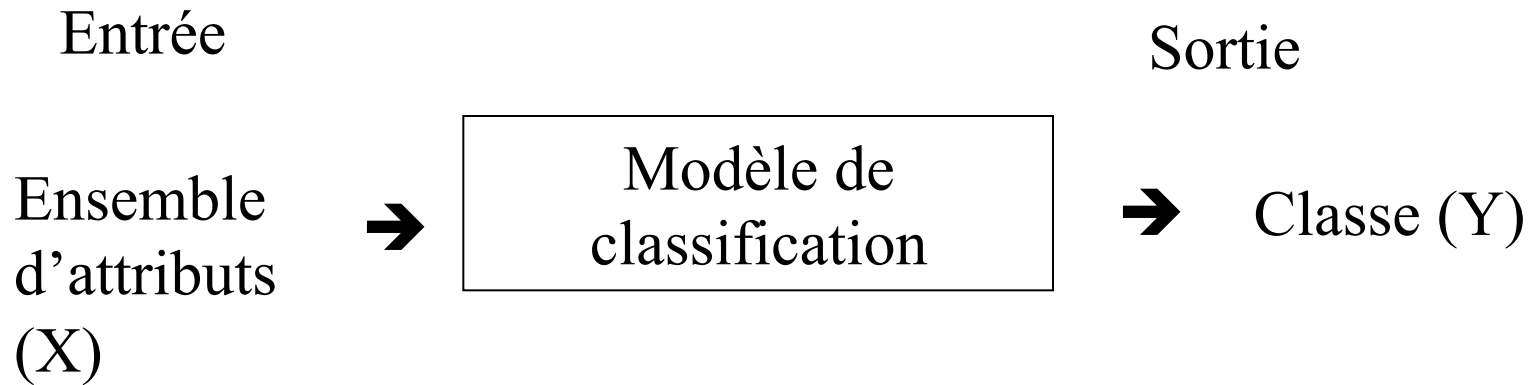
CLASSIFICATION

- Classification
 - Définitions et concepts
 - Approche générale de construction de modèles
 - Evaluation de la performance du modèle
- Arbres de décision
 - Le modèle Arbre de décision
 - Algorithme de construction
 - Définition d'une partition à partir d'un attribut
 - Choix de l'attribut et de la partition
 - Critères d'arrêt de partitionnement
 - Elagage de l'arbre

Classification

- Une technique importante de l'apprentissage automatique
- Affectation d'objets à un ensemble de catégories prédéfinies
- Permet de décrire des tendances dans les relations au sein d'un ensemble de données
- Domaines d'applications
 - Diagnostic médical, classification de plantes, techniques de marketing,...

Classification



Classification

- Modèle de classification : fonction f qui affecte chaque objet (individu) à une classe prédéfinie
- Aspect descriptif : Modèle utilisé pour décrire des objets (enregistrements) différents
- Aspect prédictif : Modèle utilisé pour prédire la classe de nouveaux objets

Classification

Nom	Age	Revenu	Côte de credit
Samba Fall	≤ 30	faible	passable
Fatou Fall	≤ 30	faible	excellente
Ibra Ndaw	31...40	élevée	excellente
Moussa Lam	>40	moyen	passable
Ndèye Lô	>40	moyen	passable
...			

Nom	Age	Revenu	Côte de crédit
Awa Dème	31...40	high	?

Classification

- Vocabulaire
 - Objets, individus, observations, enregistrements
 - X : variables descriptives, variables explicatives, variables prédictives, descripteurs
 - Y : variable à expliquer, variable à prédire
- Techniques de classification plus dédiées à décrire ou prédire des données binaires ou nominales (qualitatives)
 - Données ordinales : ne prennent pas en compte l'ordre

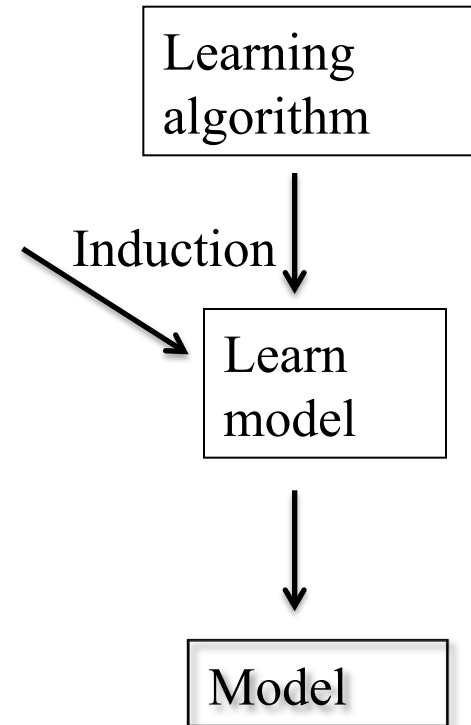
Approche générale de construction de modèles

- Classifieur : approche systématique pour la construction de modèles de classification à partir d'un ensemble de données
- Différents types de classifieurs :
 - Arbres de décision
 - Classifieurs à base de règles
 - Réseaux neuronaux
 - Support Vector Machine (SVM)
 - Réseaux bayésiens naïfs

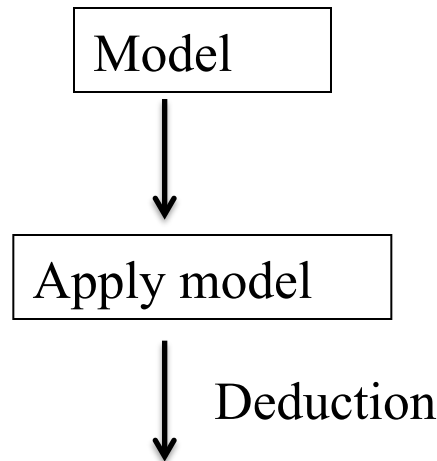
Approche générale de construction de modèles

Training set

Tid	Attrb1	Attrib2	Attrib3	Class
1	Yes	large	125K	No
2	No	medium	100K	No
3	No	small	70K	No
4	Yes	medium	120K	No
5	No	large	95K	Yes
6	No	medium	60K	No
7	Yes	large	220K	No
8	No	small	85K	Yes
9	No	medium	75K	No
10	No	small	90K	Yes



Approche générale de construction du modèle



Tid	Attrib1	Attrib2	Attrib3	Class
11	No	small	55K	?
12	Yes	medium	80K	?
13	Yes	large	110K	?
14	No	small	95K	?
15	No	large	67K	?

Test set

Evaluation de la performance du modèle

- Basé sur le comptage des enregistrements correctement et incorrectement prédits par le modèle
- Comptage effectué par la matrice de confusion
- Matrice de confusion : outil de mesure de la qualité d'un système de classification
 - Colonnes : nombre d'occurrences d'une classe estimée
 - Lignes : nombre d'occurrences d'une classe réelle

Matrice de confusion

Principe : confronter la vraie valeur avec la prédite

		Classe prédite	
		Classe = 1	Classe = 0
Classe réelle	Classe = 1	f_{11}	f_{10}
	Classe = 0	f_{01}	f_{00}

f_{ij} - nombre d'individus de la classe i prédits de classe j

Matrice de confusion

Prédite

Observée

	+	-	Total
+	a	b	a+b
-	c	d	c+d
Total	a+c	b+d	n

Mesures (indicateurs)

- Vrais positifs (VP) = a
- Faux positifs (FP) = c
- Taux d'erreur : $(c+b)/n$

Matrice de confusion

- Mesures (suite)
 - Sensibilité = Rappel = Taux de VP = $a / (a+b)$
 - Précision = $a / (a+c)$
 - Taux de FP = $c / (c+d)$
 - Spécificité = $d / (c+d) = 1 - \text{Taux de FP}$

Comparaison de deux méthodes d'apprentissage

	+	-	Total
+	40	10	50
-	20	30	50
Total	60	40	100

	+	-	Total
+	20	30	50
-	0	50	50
Total	20	b+d	100

Calculer les indicateurs synthétiques et comparer

Découpage des données

- Données d'apprentissage
 - Utilisées par les algorithmes d'apprentissage pour générer les classifieurs
- Données d'optimisation ou de validation
 - utilisées pour optimiser les paramètres de ces classifieurs
- Données de test
 - Utilisées pour calculer les taux d'erreur sur les classifieurs optimisés

Ensemble test

- Pour prédire les performances d'un classifieur sur les données non observées, on a besoin de données non utilisées pour la construction de ce classifieur
 - L'ensemble test doit être indépendant de l'ensemble d'apprentissage
 - Les deux ensembles doivent avoir des exemples représentatifs du problème

Prédiction des performances

- Quelques méthodes
 - Hold-out
 - Hold-out stratifié
 - Hold-out répété
 - Validation croisée
 - Leave-one-out
 - ...

Hold-out

- Un sous-ensemble pour le test, le reste pour l'apprentissage/validation (1/3, 2/3)
- Stratifié : équilibre des classes respecté
- Répété : l'erreur est la moyenne sur plusieurs hold-out
- Dilemme
 - Pour obtenir un bon classifieur, utiliser le plus de données possible
 - Pour avoir une bonne estimation de l'erreur, utiliser le plus de données de test possible

k-validation croisée (k-fold cross validation)

- Décider du nombre k fixe de partitions des exemples
- Couper les données en k partitions égales
- Utiliser une partition pour le test, les $k-1$ autres pour l'apprentissage
- Répéter le processus k fois (par permutation circulaire)
- Erreur = moyenne des erreurs sur les k partitions

k-validation croisée (k-fold cross validation)

- Souvent stratifiée
- 10 partitions très utilisées en pratique (bon compromis entre le nombre d'exemples pour l'apprentissage et pour le test)

Leave-one-out

- Cas particulier de la k-validation croisée où $k =$ nombre d'exemples
- Avantage : chaque itération utilise un nombre élevé de données pour la phase d'apprentissage
- Inconvénient : algorithme d'apprentissage exécuté n fois
- Utilisé quand $n < 100$

Sur-apprentissage (Overfitting)

- Lorsque le modèle a de mauvaises performance de prédiction
- Critère utilisé pour l'apprentissage du modèle $\neq$ critère utilisé pour efficacité du modèle
- Maximisation de la performance lors de l'apprentissage $\rightarrow$ minimisation de la performance pour efficacité du modèle

Les arbres de décision

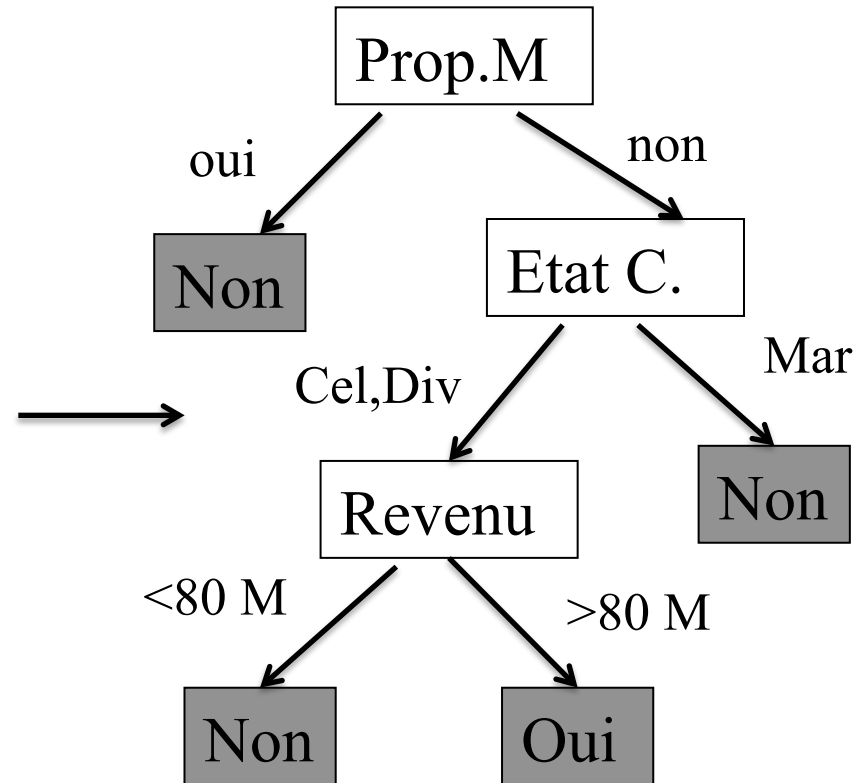
Apprentissage d'arbres de décision

- Une des formes les plus simples d'apprentissage, mais tout de même une de celles qui connaissent le plus de succès.
- A partir d'exemples, le but est d'apprendre des structures d'arbres permettant de prendre des décisions.
- Chaque nœud représente un test à faire.
- Chaque branche représente une valeur possible résultant du test.
- Une feuille correspond à la décision

Exemple d'arbre de décision

Id	Propri. maison	État civil	Revenu annuel	Prêt
1	Oui	Céli	125 M	Non
2	Non	Mar.	100 M	Non
3	Non	Céli	70 M	Non
4	Oui	Mar.	120 M	Non
5	Non	Div.	95 M	Oui
6	Non	Mar.	60 M	Non
7	Oui	Div.	220 M	Non
8	Non	Céli	85 M	Oui
9	Non	Mar.	75 M	Non
10	Non	Céli	90 M	Oui

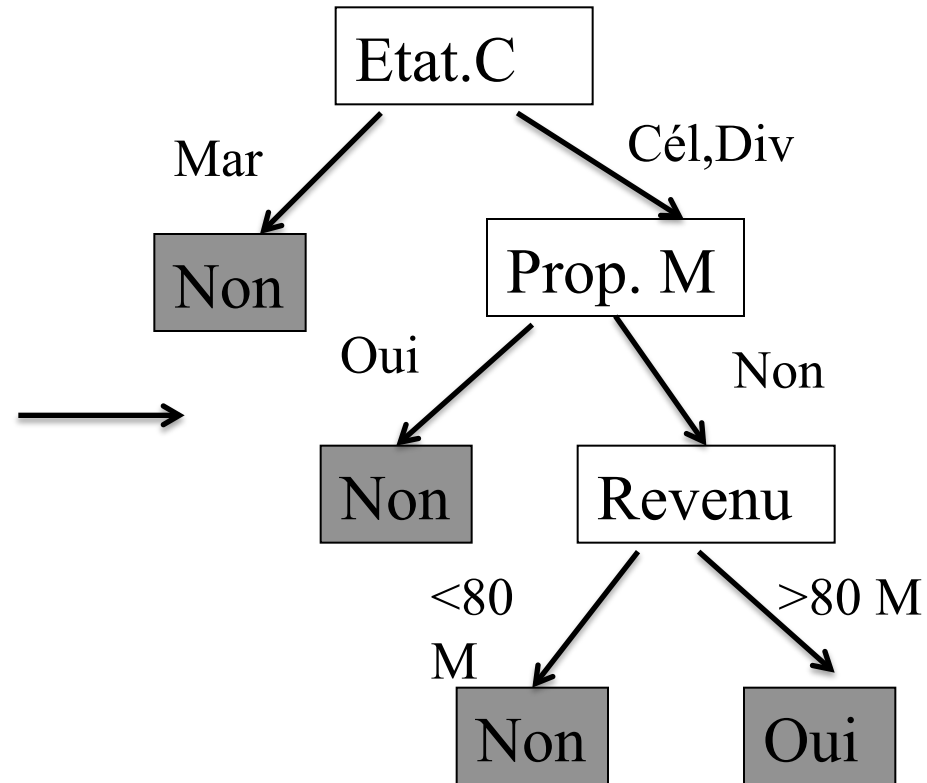
Ensemble d'apprentissage



Modèle : arbre de décision

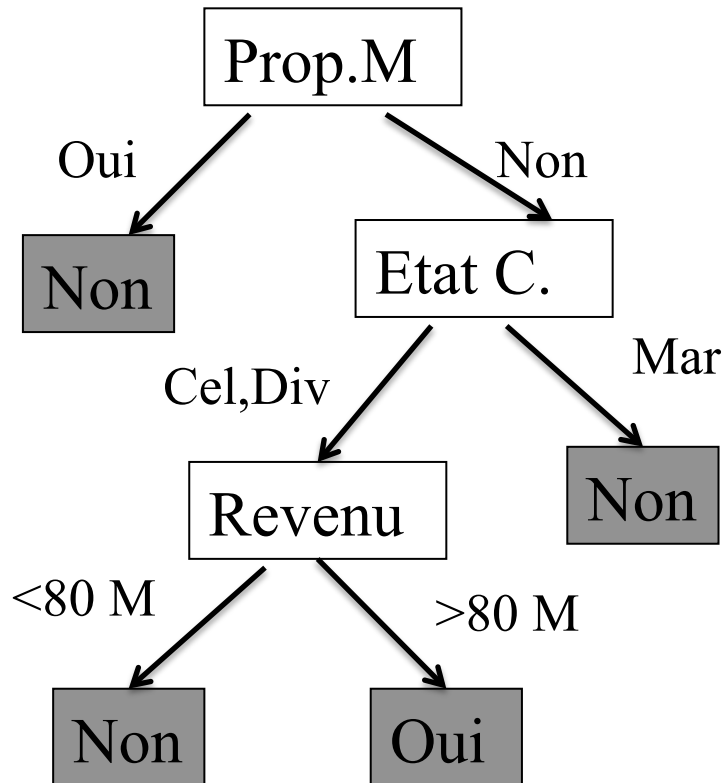
Autre exemple d'arbre

Id	Propri. maison	État civil	Revenu annuel	Prêt
1	Oui	Céli	125 M	Non
2	Non	Mar.	100 M	Non
3	Non	Céli	70 M	Non
4	Oui	Mar.	120 M	Non
5	Non	Div.	95 M	Oui
6	Non	Mar.	60 M	Non
7	Oui	Div.	220 M	Non
8	Non	Céli	85 M	Oui
9	Non	Mar.	75 M	Non
10	Non	Céli	90 M	Oui



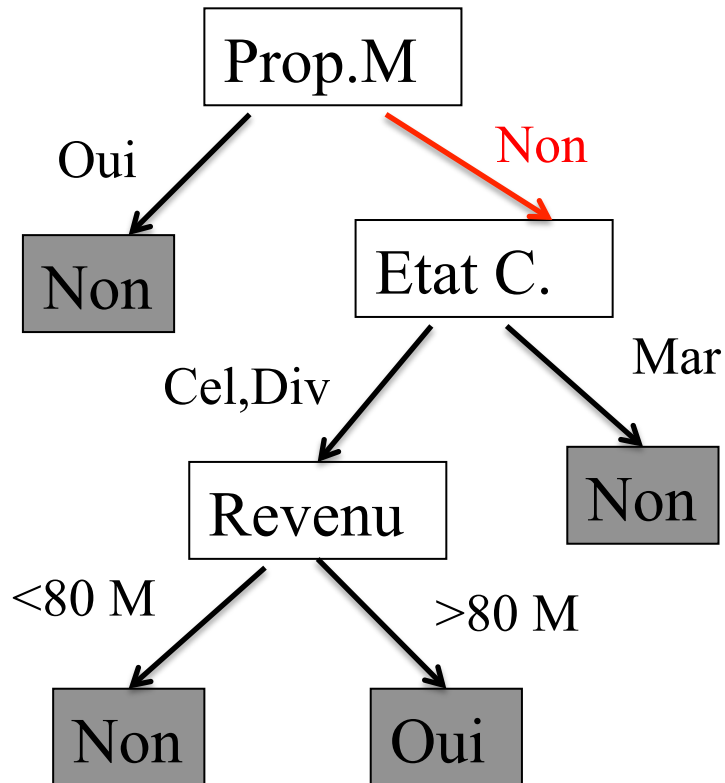
On peut avoir plusieurs arbres possibles

Application à l'ensemble test



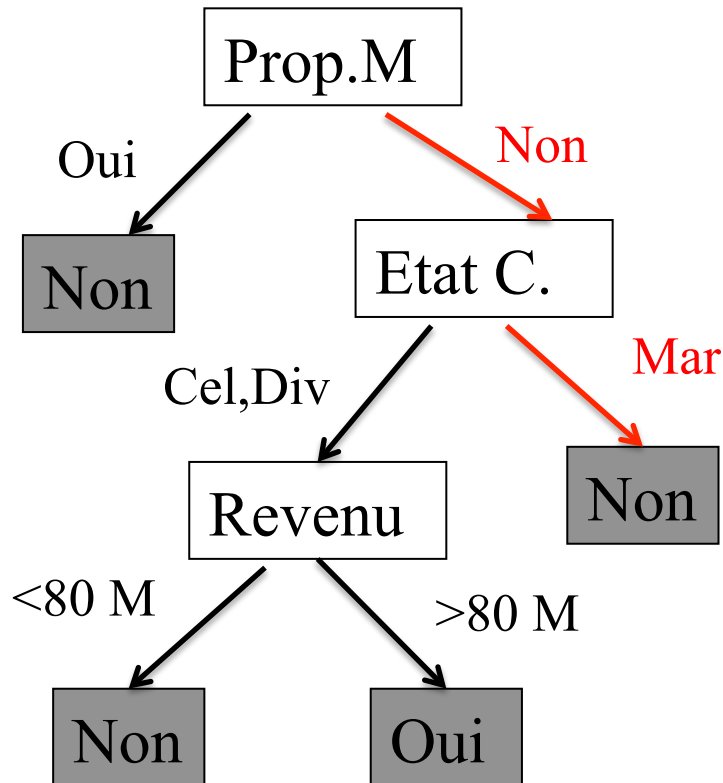
Prop M	Etat C.	Revenu annuel	Prêt
Non	Mar	80 M	?

Application à l'ensemble test

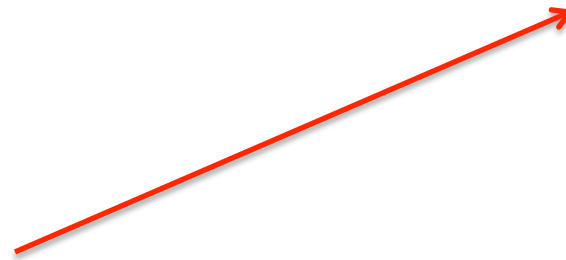


Prop M	Etat C.	Revenu annuel	Prêt
Non	Mar	80 M	?

Application à l'ensemble test



Prop M	Etat C.	Revenu annuel	Prêt
Non	Mar	80 M	?



Apprentissage d'arbres de décision

- Les arbres de décision sont des arbres qui permettent, la plupart du temps, de donner une réponse binaire comme oui ou non.
- Les instances sont représentées par des pairs (attribut, valeur). Les valeurs peuvent être discrètes ou continues
- La fonction visée est généralement binaire, mais il est facilement possible d'étendre les arbres de décision pour qu'ils puissent avoir plusieurs valeurs en sortie.
- Ils pourraient donner en sortie des valeurs réelles, mais ce n'est pas beaucoup utilisé.

Apprentissage d'arbres de décision

- Les exemples d'entraînement peuvent contenir des erreurs
 - Les erreurs dans les exemples d'entraînement sont souvent appelées des bruits.
 - Les arbres de décision sont robustes aux erreurs de classification et aux erreurs dans les valeurs des attributs.
 - Bien sûr, il ne faut pas qu'il y en ait trop
- Il peut manquer des valeurs pour certains attributs dans les exemples d'entraînement
- Les arbres de décisions sont bons pour les problèmes de classification

Construire un arbre de décision

- L'apprentissage d'un arbre de décision est fait à partir d'exemples de valeurs d'attributs et de la valeur résultante du prédicat à apprendre.
- La valeur du prédicat est appelée la classification de l'exemple (ex : Vrai / Faux).
- L'ensemble des exemples est appelé l'ensemble d'entraînement.
- Pour un même ensemble d'entraînement, on peut avoir plusieurs arbres possibles.

Construire un arbre de décision

- But : trouver le plus petit arbre qui respecte l'ensemble d'entraînement.
- Il ne s'agit pas uniquement de mémoriser les observations : il faut trouver un arbre capable d'extrapoler pour les exemples qu'il n'a pas déjà vu.
- L'arbre doit extraire des tendances ou des comportements à partir des exemples

Principes de construction

- Données de départ
 - Un ensemble de K classes $C_1, C_2, \dots, C_K$
 - Un ensemble d'exemples décrits selon $N+1$ attributs ou variables
 - N variables descriptives (douleur, fièvre,...)
 - 1 variable catégorielle : à quelle classe appartient un objet donné (rhume, appendicite,...)
- Algorithme
 - Par partitionnement successif de l'ensemble de données
 - En ensemble de données plus petits, mais plus homogènes

Algorithme

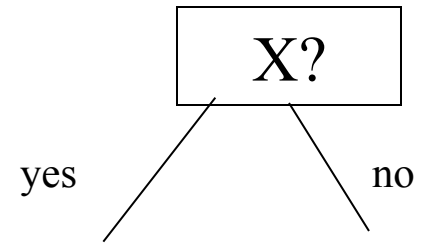
- Construction de haut en bas (top-down).
 - Place à la racine l'attribut le plus important, c'est à dire celui qui sépare le mieux les exemples positifs et négatifs.
 - Par la suite, il y a un nouveau nœud pour chacune des valeurs possibles de cet attribut.
 - Pour chacun de ces nœuds, on recommence le test avec le sous-ensemble des exemples d'entraînement qui ont été classés dans ce nœud .
- Élagage ascendant de l'arbre (Bottom-Up)
 - Enlever les sous-arbres ou branches, de manière ascendante, pour améliorer la précision estimée sur les nouveaux cas

Principales questions

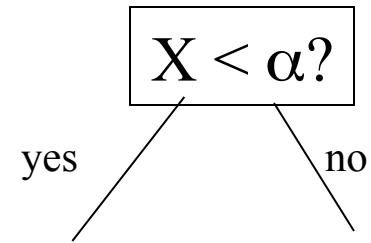
- Comment définir?
 - Une partition à partir d'un attribut
- A chaque étape
 - Quel attribut et quelle partition choisir?
- Selon quel critère
 - Arrêter de partitionner un ensemble
- Comment réduire?
 - La taille et la complexité d'un arbre

Définition d'une partition

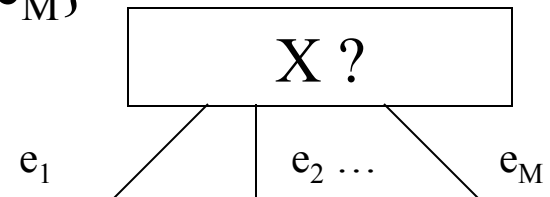
- Variable binaire $X \in \{\text{yes}, \text{no}\}$



- Variable quantitative X



- Variable qualitative X
 - Avec M modalités dans $E = \{e_1, \dots, e_M\}$



Choix d' une partition

- Principe
 - Partitionner les données en sous-ensembles les plus « purs » possibles, i.e. contenant des exemples d' une seule classe
- Différents critères d' impureté
 - Pour tout ensemble de données S partitionné en K classes $C_1, C_2, \dots, C_K$
 - Soit $P_k = |C_k| / |S|$ pour $k=1, \dots, K$
 - Taux de mal classés
 - $I(S) = 1 - \max_k(P_k)$

Choix d' une partition

- Différents critères d' impureté (suite)
 - **Entropie**
 - $I(S) = - \sum_k P_k * \log_2 P_k$
 - Critère de Gini
 - $I(S) = 1 - \sum_k P_k * P_k$

Gain d'information

- Définition

$$\text{Gain}(S, A) = \text{Entropie}(S) - \sum_{v \in V(A)} (|S_v| / |S|) * \text{Entropy}(S_v)$$

S : les exemples d'entraînement.

A : l'attribut à tester.

V(A) : les valeurs possibles de l'attribut A.

S_v le sous-ensemble de S qui contient les exemples qui ont la valeur v pour l'attribut A.

- Le gain d'information calcule la réduction d'entropie qui résulte de la partition des données selon l'attribut A.

Exemple

- Est-ce une bonne journée pour jouer au tennis?

<i>Journée</i>	<i>Ciel</i>	<i>Température</i>	<i>Humidité</i>	<i>Vent</i>	<i>JouerTennis</i>
J1	Ensoleillé	Chaude	Élevée	Faible	Non
J2	Ensoleillé	Chaude	Élevée	Fort	Non
J3	Nuageux	Chaude	Élevée	Faible	Oui
J4	Pluvieux	Tempérée	Élevée	Faible	Oui
J5	Pluvieux	Froide	Normal	Faible	Oui
J6	Pluvieux	Froide	Normal	Fort	Non
J7	Nuageux	Froide	Normal	Fort	Oui
J8	Ensoleillé	Tempérée	Élevée	Faible	Non

Exemple

<i>Journée</i>	<i>Ciel</i>	<i>Température</i>	<i>Humidité</i>	<i>Vent</i>	<i>JouerTennis</i>
J9	Ensoleillé	Froide	Normal	Faible	Oui
J10	Pluvieux	Tempérée	Normal	Faible	Oui
J11	Ensoleillé	Tempérée	Normal	Fort	Oui
J12	Nuageux	Tempérée	Élevée	Fort	Oui
J13	Nuageux	Chaude	Normal	Faible	Oui
J14	Pluvieux	Tempérée	Élevée	Fort	Non

Exemple

- Premièrement, il faut choisir la racine de l'arbre.
- Pour cela, choisir l'attribut qui a le plus grand gain d'information.
 - Pour calculer le gain d'information, il faut d'abord calculer l'entropie des exemples d'entraînement.
 - Il y a 9 exemples positifs et 5 exemples négatifs, donc on obtient une entropie de :

$$\begin{aligned} \text{Entropie}(S) &= \sum_{i=1..c} (-p_i) \log_2 (p_i) \\ &= (-9/14) \log_2 (9/14) + (-5/14) \log_2 (5/14) \\ &= 0.94 \end{aligned}$$

Exemple

- Maintenant, calculer le gain d'information pour le premier attribut, l'attribut *Ciel*.
 - Cet attribut a 3 valeurs possibles, donc les exemples d'entraînement seront regroupés en 3 sous-ensembles.
 - Commençons par calculer l'entropie des 3 sous-ensembles :

$$\text{Entropie}(S_{\text{Ensoleillé}}) = (-2/5) \log_2 (2/5) + (-3/5) \log_2 (3/5) = 0.971$$

$$\text{Entropie}(S_{\text{nuageux}}) = (-4/4) \log_2 (4/4) + (-0/4) \log_2 (0/4) = 0$$

$$\text{Entropie}(S_{\text{pluvieux}}) = (-3/5) \log_2 (3/5) + (-2/5) \log_2 (2/5) = 0.971$$

Exemple

- Le calcul du gain d'information pour l'attribut *Ciel* va donc donner :

$$\begin{aligned} \text{Gain}(S, \text{Ciel}) &= \text{Entropie}(S) - \sum_{v \in V(\text{Ciel})} (|S_v| / |S|) * \\ &\quad \text{Entropie}(S_v) \\ &= 0.94 - ((5/14) * \text{Entropie}(S_{\text{ensoleillé}}) + (4/14) * \\ &\quad \text{Entropie}(S_{\text{nuageux}}) + (5/14) * \text{Entropie}(S_{\text{pluvieux}})) \\ &= 0.94 - ((5/14) * 0.971 + (4/14) * 0 + (5/14) * \\ &\quad 0.971) \\ &= 0.94 - 0.694 \\ &= 0.246 \end{aligned}$$

Exemple

- On calcule le gain de la même manière pour les trois autres attributs :

$$Gain(S, Ciel) = 0.246$$

$$Gain(S, Humidité) = 0.151$$

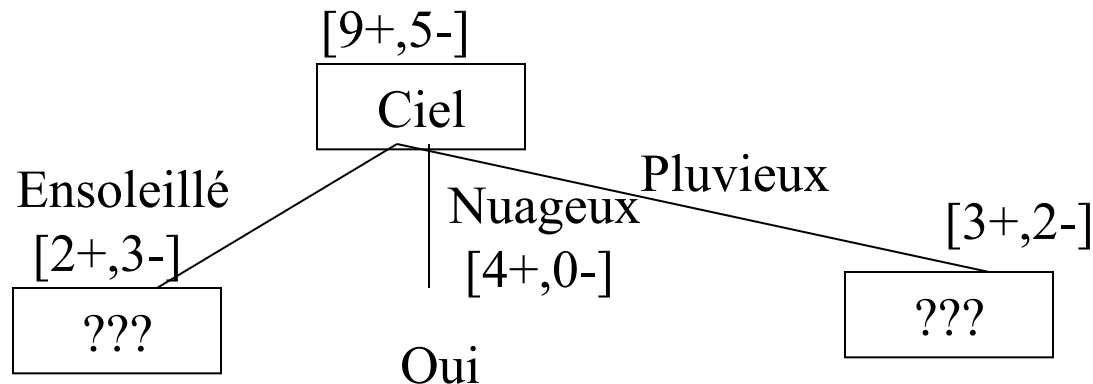
$$Gain(S, Vent) = 0.048$$

$$Gain(S, Température) = 0.029$$

- L'attribut qui a le plus grand gain d'information est l'attribut *Ciel*, donc ce sera la racine de l'arbre de décision.

Exemple

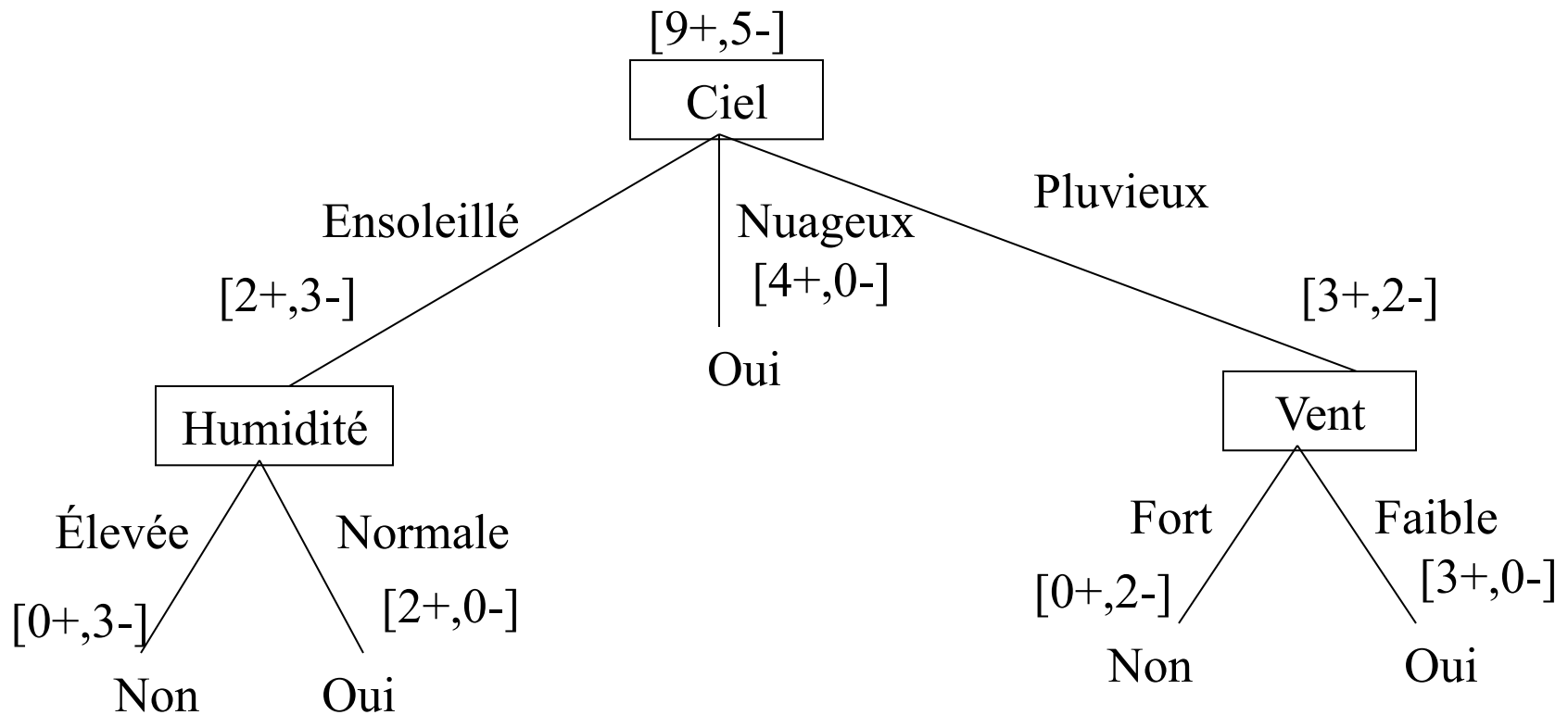
En séparant les exemples selon les valeurs de l'attribut *Ciel*, on obtient l'arbre partiel :



- On peut voir que lorsque le ciel est nuageux, il reste uniquement des exemples positifs, donc ce nœud devient une feuille avec une valeur de Oui pour la fonction visée.
- Pour les deux autres nœuds, il y a encore des exemples positifs et négatifs, alors il faut recommencer le même calcul du gain d'information, mais avec les sous-ensembles restants

Exemple

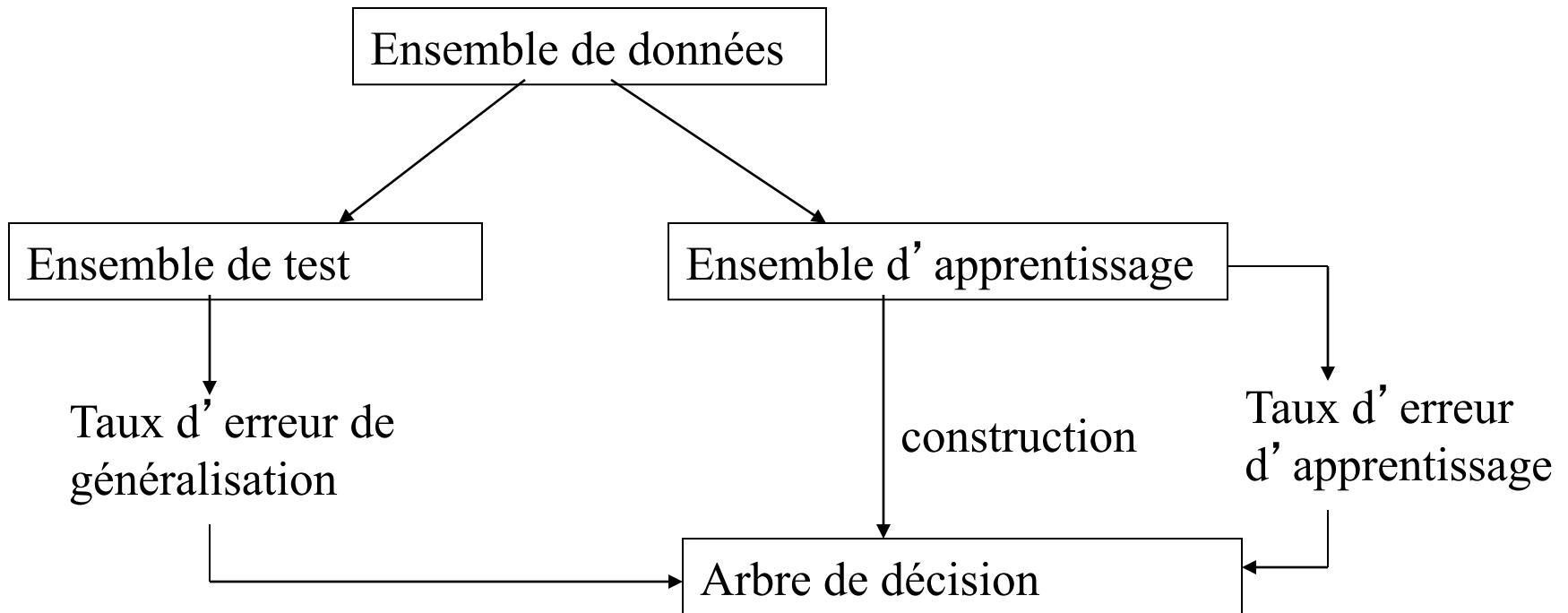
En effectuant les calculs restants, on obtient :



Gérer les erreurs dans les exemples

- Si les exemples d'entraînement contiennent des erreurs, il se peut qu'on trouve un arbre plus grand, mais qui ne classifiera pas bien les instances autres que celles utilisées dans les exemples d'entraînement
- Pour vérifier notre arbre, on peut utiliser un ensemble de validation .
 - L'ensemble de validation est généralement le tiers des exemples disponibles pour l'apprentissage

Taux d'erreurs



Sur-apprentissage

- Problème posé
 - L' algorithme peut se poursuivre de telle manière que toutes les feuilles soient pures (avec au pire un objet par feuille)
 - Taux d' erreur nul sur l' ensemble d' apprentissage
 - Taux d' erreur potentiellement élevé sur l' ensemble de test
- Solutions possibles
 - Minimiser l' erreur de généralisation en réduisant la complexité de l' arbre généré (nombre de feuilles)
 - Règles d' arrêt (pre-pruning)
 - Élagage (post-pruning)

Règles d'arrêt

- Les plus simples
 - Nbre d'éléments dans une feuille inférieur à un seuil
 - Taux de bien classés dans une feuille supérieur à un seuil
 - Exemple : $\max_k(P_k) > 95\%$
 - Gain d'information inférieur à un seuil
- Autre approche : test du Chi2
 - Approche statistique permettant de tester s'il est encore pertinent de découper un nœud
 - Principe : étudier la dépendance entre les attributs du nœud et la classe

Exemple (test du Chi2)

- Soit X : température , Y : JouerTennis

X \ Y	oui	non	Marge
chaude	2	2	4
froide	3	1	4
tempérée	4	2	6
Marge	9	5	14

X \ Y	oui	non
chaude	$\frac{4*9}{14}$	$\frac{4*5}{14}$
froide	$\frac{4*9}{14}$	$\frac{4*5}{14}$
tempérée	$\frac{6*9}{14}$	$\frac{6*5}{14}$



Tableau attendu s' il y a indépendance

Exemple (test du Ki2)

- $Ki^2 = (o_1 - e_1)^2 / e_1 + (o_2 - e_2)^2 / e_2 + \dots + (o_n - e_n)^2 / e_n$
- Si Ki^2 est très grand, alors les attributs X et Y ne sont pas indépendants $\rightarrow$ i.e., ils sont dépendants
- Comparaison avec la valeur critique c qui dépend du nbre de degré de liberté v .
 - Pour une table de $n*m$ items, $v = (n-1)*(m-1)$
 - Trouver la valeur de c en fonction de v dans le tableau de Ki^2
- Si tous les attributs d'un nœud sont indépendants par rapport à la classe, alors on arrête le découpage du nœud.
- ID3 utilise le test du chi2 en plus du gain d'information
 - Seuls les attributs statistiquement significatifs sont pris en compte dans le calcul du gain

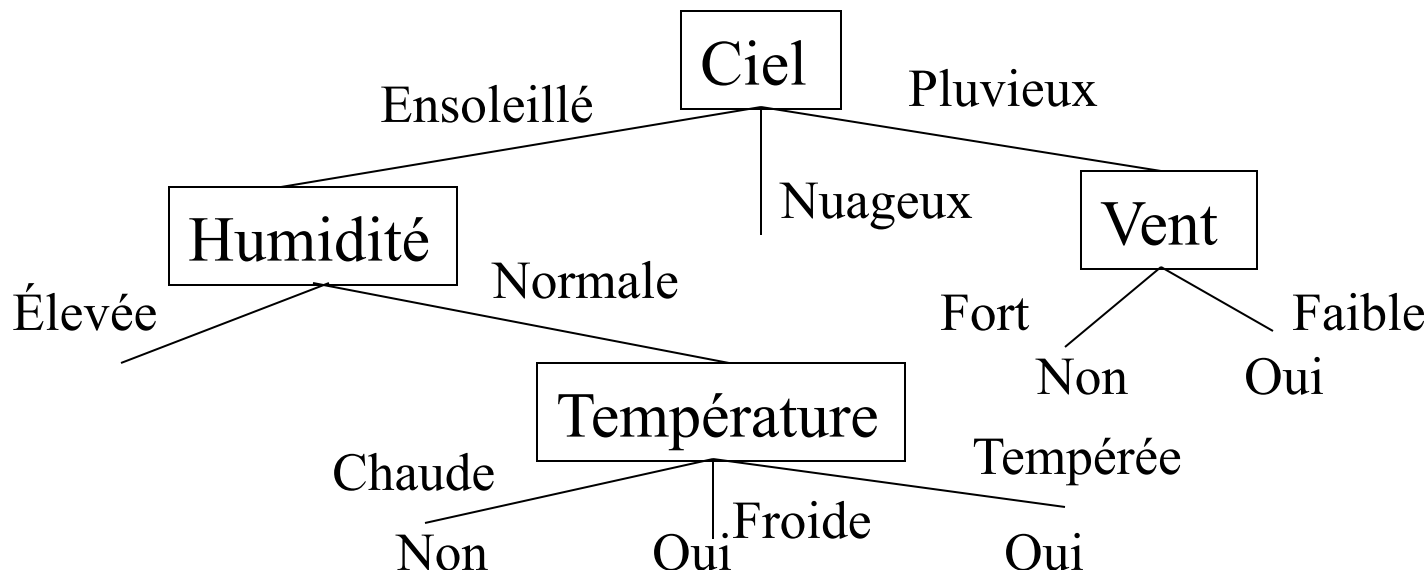
Élagage de l' arbre

- Cette méthode coupe des parties de l' arbre en choisissant un nœud et en enlevant tout son sous-arbre.
 - Ceci fait donc du nœud une feuille et on lui attribut la valeur de classification qui revient le plus souvent
- Des nœuds sont enlevés seulement si l' arbre résultant n' est pas pire que l' arbre initial sur les exemples de validation
- On continue tant que l' arbre résultant offre de meilleurs résultats sur les exemples de validation
- Ceci a pour but de réduire l' arbre en enlevant des branches qui auraient été ajoutées par une erreur dans les exemples.

Élagage de l'arbre

- Si on ajoute une erreur dans l'exemple précédent en modifiant le premier exemple, obtient l'arbre suivant :

J1	Ensoleillé	Chaude	Élevée	Faible	Non
J2	Ensoleillé	Chaude	Normale	Faible	Non



Élagage de l' arbre

- L' élagage pour cet arbre consiste à enlever le nœud *Température* qui vient d' être ajouté avec l' erreur
- Par la suite, on teste le nouvel arbre élagué sur les exemples de validation.
- Si la performance n' est pas diminuée, alors on conserve l' arbre élagué
- C' est par la suite que cet arbre va être utilisé pour l' application à l' ensemble test
- Si pas du post-pruning, mais seulement du pre-pruning, un tel découpage de l' ensemble d' apprentissage (en jeu de d' apprentissage et de validation) n' est pas nécessaire

Propriétés des algorithmes de construction

- Pour être utile dans la plupart des applications réelles, un algorithme doit :
 - Prendre en compte des attributs numériques
 - Prendre en compte des valeurs manquantes
 - Être robuste à la présence de bruits (erreurs de classification, erreurs dans les valeurs des attributs)

Valeurs d'attributs manquantes

- S'il y a des valeurs pour certains attributs qui ne sont pas disponibles, alors on peut :
 - Donner la valeur moyenne pour cet attribut.
 - On regarde les autres exemples et on calcule la moyenne des valeurs présentes.
 - On utilise cette moyenne pour estimer la valeur manquante dans le calcul du gain d'information
 - Attribuer une probabilité pour la valeur manquante.
 - On regarde les autres exemples et on calcule la probabilité de chaque valeur possible pour l'attribut
 - On utilise par la suite ces probabilités pour calculer le gain d'information.

Valeurs d'attributs manquantes

- Pour la première stratégie, les calculs ne changent pas. On ne fait qu'utiliser la valeur moyenne pour remplacer la valeur manquante.
- Pour la deuxième stratégie, les calculs sont un petit peu modifiés pour utiliser les probabilités.
 - On commence par calculer la probabilité de chacune des valeurs possibles pour l'attribut manquant
 - Par exemple, supposons qu'il nous manquerait une valeur pour l'attribut *Vent*.
 - La probabilité que le vent soit faible, selon nos exemples d'entraînement est de $8/14 * 100 = 57\%$.
 - La probabilité que le vent soit fort est de $6/14 * 100 = 43\%$
 - Ceci va nous donner des fractions d'exemples dans nos calculs.

Exemple

- Supposons que l'on a un exemple d'entraînement *positif* de plus, mais que la valeur pour l'attribut *Vent* est absente.

$$\begin{aligned} \text{Entropie}(S) &= \sum_{i=1..c} -p_i \log_2 p_i \\ &= -10/15 \log_2 10/15 + (-5/15 \log_2 5/15) \\ &= 0.918 \end{aligned}$$

$$\text{Entropie}(S_{\text{fort}}) = (-4/6.43) \log_2 4/6.43 + (-3/6.43) \log_2 3/6.43 = 0.939$$

$$\text{Entropie}(S_{\text{faible}}) = (-7/8.57) \log_2 7/8.57 + (-2/8.57) \log_2 2/8.57 = 0.728$$

$$\begin{aligned} \text{Gain}(S, \text{Vent}) &= \text{Entropie}(S) - \sum_{v \in V(\text{Vent})} (|S_v| / |S|) \text{Entropie}(S_v) \\ &= 0.918 - ((6.43/15) * \text{Entropie}(S_{\text{fort}}) + (8.57/15) * \\ &\quad \text{Entropie}(S_{\text{faible}})) \\ &= 0.918 - ((6.43/15) * 0.939 + (8.57/15) * 0.728) \\ &= 0.918 - 0.818 \\ &= 0.1 \end{aligned}$$

Attributs multi-valués

- Il y a un petit problème avec la fonction de gain d'information.
 - Lorsque les attributs ont beaucoup de valeurs possibles, comme par exemple un attribut *date*, leur gain est très élevé, car il classifie parfaitement les exemples
 - Par contre, ils vont générer un arbre de décision d'une profondeur de 1 qui ne sera pas très bon pour les instances futures
- Solution : utiliser une fonction *GainRatio* qui pénalise les attributs qui ont trop de valeurs possibles.

Attributs multi-valués

- On utilise la fonction *GainRatio* au lieu *Gain* lorsque l'on a des attributs multivalués.

$$GainRatio(S,A) = Gain(S,A) / SplitInformation(S,A)$$

$$SplitInformation(S,A) = - \sum_{i=1..c} (|S_i| / |S|) \log_2 (|S_i| / |S|)$$

S_1 à S_c sont les c ensembles résultant de la partition de S par les c valeurs de l'attribut A .

Attributs à valeurs continues

- On utilise un point de coupe pour obtenir une discrétisation des variables continues.
 - Ex: la variable *Température* est continue et on a les 6 exemples suivants.

Température	40	48	60	72	80	90
JouerTennis	Non	Non	Oui	Oui	Oui	Non

- On met les valeurs en ordre croissant et on regarde les endroits où la fonction change de valeur.
- A ces endroits, on choisit la médiane comme valeur de coupe
- On compare toutes les valeurs de coupe et on choisit celle qui apporte le plus grand gain d'information.

Exemple

Température	40	48	60	72	80	90
JouerTennis	Non	Non	Oui	Oui	Oui	Non

- Il y a deux valeurs de coupe :
 - $(48 + 60) / 2 = 54$
 - $(80 + 90) / 2 = 85$
- On calcule les gains d'information pour ces deux valeurs de coupe

$$\begin{aligned} \text{Entropie}(S) &= \sum_{i=1..c} -p_i \log_2 p_i \\ &= -3/6 \log_2 3/6 + (-3/6 \log_2 3/6) \\ &= 1 \end{aligned}$$

Exemple

- Gain pour la coupe 54 :

$$\begin{aligned} \text{Gain}(S, T_{54}) &= \text{Entropie}(S) - \sum_{v \in V(T_{54})} (|S_v| / |S|) \text{Entropie}(S_v) \\ &= 1 - (((2/6) * (-2/2 \log_2 2/2 + -0/2 \log_2 0/2)) + \\ &\quad ((4/6) * (-3/4 \log_2 3/4 + -1/4 \log_2 1/4))) \\ &= 1 - (0 + 0.541) \\ &= 0.459 \end{aligned}$$

- Gain pour la coupe 85

$$\begin{aligned} \text{Gain}(S, T_{85}) &= \text{Entropie}(S) - \sum_{v \in V(T_{85})} (|S_v| / |S|) \text{Entropie}(S_v) \\ &= 1 - (((5/6) * (-2/5 \log_2 2/5 + -3/5 \log_2 3/5)) + \\ &\quad ((1/6) * (-1/1 \log_2 1/1 + -0/1 \log_2 0/1))) \\ &= 1 - (0.809 + 0) \\ &= 0.191 \end{aligned}$$

Exemple

- En comparant les gains d'information, on se rend compte que 54 est la meilleure valeur de coupe. Par conséquent, la température sera utilisée comme un attribut binaire qui ne pourra prendre que 2 valeurs : plus petit que 54 ou plus grand ou égale à 54.

Bagging

- Méthode générale
 - Pour améliorer la stabilité d' une classification
- Principe de base
 - Découper l' ensemble d' apprentissage en P échantillons $E_1, \dots, E_P$
 - Construire un arbre de décision B_j (ou autre modèle) à partir de chaque échantillon E_j
 - Combiner les modèles obtenus pour prendre des décisions
 - Par un vote majoritaire : la classe de l' objet considéré est la classe la plus souvent prédite par les modèles
 - Par le calcul des probabilités

Principaux algorithmes

- ID3 (Quinlan, 1979)
- C4.5 (Quinlan, 1986), amélioration de ID3
 - Prise en compte des attributs numériques
 - Valeurs manquantes
 - Gestion du bruit
- CART (Breiman, Friedman, Ohlson, Stone, 1984)
- C4.5 est l'un des algorithmes les plus connus et les plus utilisés (J48 – version Weka de C4.5)